

Laboratorio 7

Aprendizaje por refuerzo

Francis Aguilar, 22243

Jose Marchena, 22398

Cesar Lopez, 22535

Repositorio

https://github.com/MarchMol/lab7_rl

Instrucciones

Una empresa de ciberseguridad opera un sistema de detección de intrusiones en redes corporativas. El sistema monitorea el tráfico de red en tiempo real y debe decidir qué acciones tomar ante comportamientos sospechosos: **ignorar, alertar, bloquear temporalmente o aislar el segmento afectado**.

El estado del sistema en cada instante es un vector de alta dimensión que incluye métricas de tráfico, patrones de comportamiento de usuarios y logs de eventos recientes. La empresa quiere explorar si **Deep RL** puede mejorar la toma de decisiones frente a los sistemas basados en reglas que usan actualmente.

Task 1 (Entrega Parcial)

Task 1.1

El equipo de ingeniería propone usar **DQN estándar** para este sistema. Antes de implementar cualquier cosa, usted debe hacer un análisis crítico de esa propuesta.

a.

El estado del sistema es un vector de **256 dimensiones con valores continuos**.

Argumenten si DQN es arquitecturalmente apropiado para este tipo de estado.

DQN si puede ser apropiado para este tipo de estado, dado que es agnostico al modelo de aprendizaje profundo que se integre. Es decir, cuando se usa para videojuegos como el caso de Atari ellos utilizaron una red convolucional sobre el input de pixeles. En nuestro caso, una red convolucional talvez no sea tan util, pero definitivamente un vector de entrada de 256 dimensiones es apropiado. Asimismo, las redes neuronales por detras manejan muy bien valores continuos y en todo caso, es una de las razones principales por las cuales utilizar DQN en lugar de otros tipos de agentes que estan limitados a espacios discretos. ¿Qué tipo de red usarían como aproximador de $Q(s, a; \mathbf{w})$ y por qué, considerando que el estado no es una imagen sino un vector numérico? Lo mas sencillo y justificable es utilizar una red completamente conectada. Esta nos permite encontrar relaciones entre las diferentes features numericas y concentrar el resultado en un solo valor ya sea con un softmax o algo por el estilo.

b.

El espacio de acciones tiene **4 opciones discretas**.

Argumenten si DQN o alguna de sus variantes es apropiado para este espacio de acciones, o si sería necesario un algoritmo diferente.

El espacio de 4 opciones discretas es apropiado e incluso ideal para todos los casos de DQN.

c.

En el dominio de detección de intrusiones, los eventos críticos (**intrusiones reales**) son extremadamente raros: ocurren en menos del **0.1% de los pasos de tiempo**.

Argumenten formalmente qué consecuencia tiene esa rareza sobre el buffer de **Experience Replay** y sobre la distribución de muestreo uniforme.

¿Qué variante de DQN resolvería este problema específico y cómo?

DQN estandar talvez no sea ideal para nuestro caso de uso dado sus problemas con estabilidad, sobreestimación e ineficacia del sampling. Se recomienda utilizar una variante ya sea Double DQN o Dueling DQN, pues nos permiten sobrepasar varias limitaciones y Dueling funciona bien para nuestro caso de uso, donde varias acciones pueden llevar a un futuro idéntico, que es el caso donde los eventos críticos son raros.

d.

El sistema actual basado en reglas tiene una tasa de falsos positivos del **8%**.

Si diseñaran la función de recompensa de DQN para minimizar falsos positivos únicamente, ¿qué comportamiento indeseable podría aprender el agente?

Si se busca minimizar falsos positivos, es posible que el modelo pueda llegar a un overfitting donde casi nunca marca como crítico a una conexión, dejando de lado todo su objetivo de identificar correctamente estas conexiones maliciosas.

Diseñen una función de recompensa con **al menos tres componentes** que capture el objetivo real del sistema, justificando la magnitud relativa de cada uno.

Aunque se desconoce la naturaleza o significado específico de las features como para armar una función de recompensa basado en el valor específico del estado, se puede abordar la función de recompensa sobre las acciones y la retroalimentación entre estados. Por tanto se proponen los siguientes 3 mecanismos principales para la recompensa:

1. Accuracy: da recompensa a las identificaciones de conexión crítica. Sin esta, el modelo nunca buscaría identificar correctamente, pero no elimina la posibilidad de falsos positivos.
- Regularización: mecanismos agresivos de regularización para penalizar acciones demasiado defensivas, esto debería disminuir la cantidad de falsos positivos.

- Penalización por acción: no todas las acciones son iguales en términos de el efecto que tienen sobre la usabilidad, entonces debe haber un descuento sobre la fricción introducida por cada acción. Las acciones de ignorar o alertar deben tener una baja o nula penalización, pero acciones fuertes como bloquear temporalmente o aislar deben tener una penalización representativa.

Con respecto a la magnitud relativa de cada una, se propone la siguiente magnitud relativa $\text{Accuracy} > \text{Fricción por acción} > \text{regularización}$. La identificación positiva debe de ser el componente de mayor influencia pues es la que mueve las acciones del modelo. Luego las fricciones por acción siguen pues acciones muy agresivas como aislamiento o bloqueo temporal tienen efectos cruciales en la usabilidad de la plataforma. Por último la regularización para alisar la continuidad de falsos positivos.

Task 1.2

El equipo propone usar **Double DQN** en lugar de DQN estándar.

a.

Expliquen formalmente el problema de **sobreestimación** que Double DQN resuelve.

El problema de sobreestimación origina de la función de target que utilizan estos agentes para calcular el valor de una acción $TARGET = R + \gamma \max_a Q(s', a)$. La toma a partir del valor de retorno máximo asociado a cada uno. Dado que los algoritmos detrás de DQN tienden a poseer ruido, escoger siempre el máximo sistemáticamente elige los picos del ruido presente en sus predicciones, es decir, tiende a sobreestimar el retorno esperado de cada acción.

Double DQN resuelve esto a partir de su abordaje con dos modelos, el Online y el Target. Esta separación busca diferenciar la evaluación entre el valor aparente de una acción actualmente (la red online) vs el valor actual de la acción (la red Target). Es decir mientras que la red online toma la decisión es susceptible al ruido, la red target lo evalúa de manera más precisa y esto tiene el efecto de cancelar la sobreestimación.

En el contexto específico de detección de intrusiones:

- ¿Qué consecuencia operacional tendría que el agente sobreestime sistemáticamente el valor de la acción "bloquear" ?

Si el agente sobreestima sistemáticamente BLOQUEAR, significa que ve el posible retorno de bloquear una conexión crítica y tendrá a escogerlo más frecuentemente frente a comportamientos extraños. Sin embargo, esto es exactamente lo que causa una alta tasa de falsos positivos e introduce alta fricción en comportamiento normal de usuarios al bloquear mas de los que necesita o puede detectar con certeza.

- ¿Y de la acción "ignorar" ?

Asimismo, si muestra un sesgo sobre IGNORAR va a tender a ejecutar esta acción más frecuentemente dependiendo de la ponderación de retorno. Esto puede tener exactamente el efecto opuesto al anterior: al ver que le da más retorno ignorar tráfico, puede llegar a obviar entrads que si muestren patrones críticos y corre el riesgo de no identificar correctamente estos escenarios.

b.

Escriban la expresión del **target de DQN estándar** y la del **target de Double DQN**, identificando con precisión qué parámetros hacen qué en cada caso.

$$Y_t^{\text{DQN}} = R_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta^-)$$

$$Y_t^{\text{DoubleDQN}} = R_{t+1} + \gamma Q\left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta^-\right)$$

¿Cuál es el único cambio de implementación entre ambos?

El unico cambio que presentan estos dos es exactamente lo que se mencionó que soluciona el problema de sobreestimación. En lugar de evaluar el mayor retorno de una acción, evaluamos el valor de la acción dentro de la red de Target (θ^-) que tuvo mayor valor en la red Online(θ).

c.

Argumenten si en este dominio específico la sobreestimación es un problema más grave en ciertos tipos de estados que en otros.

Consideren:

- Estados con **alta ambigüedad** (tráfico *borderline*).
- Estados **claramente maliciosos**.

Este es exactamente el problema con el que lidian detectores de tráfico malicioso. Dado que este tráfico es bastante inusual, es decir con baja probabilidad como vimos de 0.1%, un agente puede fácilmente colapsar en predicciones más estáticas sin realmente aprender que diferencia una conexión legítima de una maliciosa. Es decir, el modelo escogera muchas veces IGNORE porque es el caso mas comun. Por otra parte, también tenemos el problema de estados ambiguos si se intenta enforzar más el uso de BLOQUEAR, pues una conexión con comportamientos ambiguos pero sea legitima se puede ver bloqueada por la predicción del agente. Un bloqueo de una transacción legítima puede afectar capacidades de negocio e introducir fricción innecesaria sobre los clientes, por lo que también es un problema grave. Por tanto, depende mucho de la lógica de negocio, si un falso positivo pesa más que un falso negativo.

Task 2 (Entrega Final)

Task 2.1

Implementen **DQN** y **Double DQN** sobre el entorno `LunarLander-v2` de Gymnasium, que tiene:

- Espacio de acción discreto con **4 acciones**.
- Estado continuo de **8 dimensiones**.

Usen este entorno como *proxy* del sistema de detección de intrusiones para validar las propiedades algorítmicas antes de escalar al dominio real.

Arquitectura de la red

La red para $Q(s, a; \mathbf{w})$ debe tener:

- Dos capas ocultas.
- **128 neuronas** por capa.
- Activación **ReLU**.

Experience Replay

- Capacidad: **50,000 transiciones**.
- Minibatch: **64 transiciones**.

Target Network

Actualizar cada **500 pasos**.

Exploración ε

Usen ε decreciente:

- Inicial: $\varepsilon = 1.0$
- Final: $\varepsilon = 0.05$
- Decaimiento durante los primeros **10,000 pasos**.

Implementación

Ambos algoritmos deben implementarse **desde cero usando PyTorch**.

No se permite usar implementaciones preconstruidas de DQN de ninguna librería de RL.

Registros

La implementación debe registrar por episodio:

1. **Recompensa total**.
2. **Valor Q promedio estimado** sobre un conjunto fijo de 100 estados de evaluación.
3. **Target promedio** usado en las actualizaciones.

Estos tres registros son necesarios para detectar sobreestimación.

Task 2.2

Con base en los resultados de la implementación, respondan:

a.

Grafiquen la **recompensa promedio por episodio** de DQN y Double DQN en la misma figura con **media móvil de 20 episodios**.

Respondan:

- ¿Cuál converge más rápido?
- ¿Cuál alcanza mayor recompensa final?
- ¿El resultado coincide con lo esperado teóricamente?

b.

Grafiquen el **valor Q promedio estimado durante el entrenamiento** para ambos algoritmos.

El valor Q de DQN debería ser sistemáticamente mayor que el de Double DQN sobre los mismos estados.

Respondan:

- ¿Observan ese patrón?
- Si no lo observan, argumenten por qué podría no ser visible en este entorno específico.

c.

Implementen ahora **Prioritized Experience Replay (PER)** sobre Double DQN, usando:

- $\alpha = 0.6$
- β creciente de 0.4 a 1.0

Comparen la velocidad de convergencia de:

- Double DQN con muestreo uniforme.
- Double DQN con PER.

¿En qué fase del entrenamiento es más visible la diferencia?

d.

Argumenten si PER sería **más o menos beneficioso** en el dominio real de detección de intrusiones comparado con LunarLander, considerando la rareza de los eventos críticos que analizaron en la **Task 1.1c**.

Apoyen su argumento con evidencia de sus experimentos.

Task 2.3

Investigación bibliográfica y dictamen técnico

a.

Busquen y lean un paper publicado entre **2022 y 2025** que aplique alguno de los siguientes algoritmos:

- DQN
- Double DQN
- Dueling DQN
- PER
- Rainbow

El paper debe aplicar el algoritmo a un problema de:

- Ciberseguridad.
- Detección de anomalías.
- Gestión de redes.

Debe provenir de una publicación como:

- *IEEE Transactions on Information Forensics and Security*
- NeurIPS
- ICML
- USENIX Security
- O similar.

Escriban un **resumen técnico de media página** que incluya:

1. El problema que resuelve.
2. Qué variante de DQN usa y por qué.
3. Los resultados principales.
4. Una reflexión sobre si las limitaciones de DQN identificadas en la **Task 1** fueron resueltas en ese trabajo.

b.

Redacten un **dictamen técnico de un párrafo** dirigido al equipo directivo de la empresa de ciberseguridad.

El dictamen debe argumentar:

- Si recomiendan proceder con DQN o alguna de sus variantes para el sistema de detección de intrusiones.
- Qué variante específica recomiendan y por qué.
- Qué pasos adicionales serían necesarios antes de un despliegue en producción.

El dictamen debe estar respaldado por evidencia de sus experimentos, no solo por argumentos teóricos.

Entregas en Canvas

1. **Documento PDF** con las respuestas a cada task.

- En la entrega parcial se espera que entreguen lo señalado.
- En la entrega final deben entregar **TODOS LOS TASK**.

2. **Archivo .ipynb o link a repositorio de GitHub.**

- No se aceptan entregas en otros medios.
- El código debe estar comentado explicando la relación con las fórmulas de las diapositivas.

Evaluación

Task	Punteo
Task 1	1.20 pts
Task 2	1.80 pts
Total	3.0 pts